Tracking Performance Drift in Large Language Models Across Successive Version Updates

Abstract

Large language models (LLMs) deployed as commercial services are updated frequently, yet the substance of these updates is rarely disclosed to downstream users. A growing body of empirical work shows that model behavior on identical prompts can shift markedly between versions, sometimes improving on one task while degrading on another within the same release cycle. This paper synthesizes the emerging literature on "performance drift" or "behavior drift" in LLMs, situating it within the older machine-learning literature on concept and data drift. We develop a taxonomy of drift sources spanning weight updates, safety fine-tuning, system-prompt changes, quantization, and infrastructure substitutions; review longitudinal measurement studies and monitoring frameworks such as LLMDrift, ChatLog, and HELM; and examine confounds that complicate drift attribution, including benchmark contamination, non-deterministic decoding, and LLM-as-judge evaluator instability. We argue that the field currently lacks standardized, versioned, contamination-resistant benchmarks and shared statistical protocols for distinguishing genuine capability regression from benign behavioral variation. We close by outlining research directions — continuous evaluation infrastructure, causal attribution methods, and disclosure norms modeled on model cards — that would allow the field to track drift with the same rigor traditionally reserved for measuring capability at a single point in time.

Keywords

large language models; performance drift; behavior drift; model versioning; concept drift; benchmark contamination; continuous evaluation; reproducibility; LLM-as-judge; catastrophic forgetting

1. Introduction

Large language models (LLMs) such as GPT-4, Claude, and Gemini are no longer static artifacts released once and left unchanged. Commercial providers routinely replace the model behind a fixed API endpoint or product name with a retrained, fine-tuned, or otherwise modified successor, often without disclosing the timing, scope, or rationale of the change. This practice sits uneasily with how these systems are used: developers build production pipelines on top of a particular observed behavior, researchers report benchmark scores as though they describe a fixed object, and end users develop prompting habits calibrated to a version of the model that may no longer exist by the time they read about it.

The consequence is what Chen, Zaharia, and Zou (2023) term LLM drift: measurable, sometimes substantial, change in a model's behavior and performance across successive versions, even when the model name and the user-facing prompt remain identical. Their study of GPT-3.5 and GPT-4 across a March 2023 and a June 2023 snapshot found that GPT-4's accuracy at identifying prime numbers fell from 84% to 51% between the two releases, while GPT-3.5 improved on the same task over the same interval; code generation, instruction-following fidelity, and formatting conventions shifted in both models as well (Chen et al., 2023). These findings, amplified widely in the technology press and corroborated by independent longitudinal projects such as ChatLog (Tu et al., 2023), turned an anecdotal complaint — that "ChatGPT is getting worse" — into a subject of empirical inquiry.

Tracking performance drift matters for several interlocking reasons. First, it is a reproducibility problem: a benchmark score reported for “GPT-4” or “Claude” in one paper may not describe the model a reader encounters six months later, because the underlying weights, safety layers, or serving infrastructure have changed beneath a stable name (Chen et al., 2023; Tu et al., 2023). Second, it is an engineering-reliability problem: production systems that depend on specific output formats, refusal behaviors, or latency characteristics can silently break when an upstream model update alters any of these properties, a risk documented both in the academic literature and in industry practitioner accounts (Chen et al., 2023). Third, it is a governance and transparency problem: without standardized disclosure analogous to model cards (Mitchell et al., 2019), external stakeholders cannot audit what changed, why, or how the change was validated before deployment. Finally, it connects to a decades-old body of work on concept drift in streaming machine learning (Gama, Žliobaitė, Bifet, Pechenizkiy, & Bouchachia, 2014), which offers formal vocabulary and detection methodology that the LLM evaluation community has only begun to adapt to the peculiarities of generative, instruction-following systems.

This paper reviews and synthesizes the literature on tracking performance drift across LLM version updates. Section 2 situates LLM drift within the broader evaluation and concept-drift literatures. Section 3 develops a taxonomy of drift mechanisms and discusses how drift manifests across task types. Section 4 surveys current monitoring approaches, benchmarks, and tooling. Section 5 discusses methodological challenges that confound drift measurement. Section 6 identifies research gaps and proposes directions for future work, and Section 7 concludes.

2. Background, Related Work, and Literature Review

2.1 Concept Drift in Classical Machine Learning

The phenomenon of a learned model's accuracy degrading because the statistical relationship between inputs and outputs changes over time predates LLMs by decades and is well studied under the name concept drift in the streaming-data and online-learning literature. Gama et al. (2014) provide the standard survey, characterizing drift as sudden, gradual, incremental, or recurring, and cataloguing detection methods (e.g., sequential error-rate monitoring, windowing techniques) and adaptation strategies (incremental retraining, ensemble reweighting, instance selection). Crucially, that literature distinguishes true concept drift — a persistent change in the underlying data-generating process — from transient noise or outliers, and it insists that adaptation algorithms be evaluated against principled statistical tests rather than impressionistic before/after comparisons (Gama et al., 2014). Much of the emerging LLM-drift literature reinvents pieces of this framework informally; explicitly importing its vocabulary and evaluation discipline is one of the opportunities this paper identifies in Section 6.

2.2 The First Systematic Studies of LLM Drift

The paper most directly responsible for establishing LLM drift as a named object of study is Chen et al. (2023), “How Is ChatGPT's Behavior Changing Over Time?” The authors evaluated the March 2023 and June 2023 snapshots of GPT-3.5 and GPT-4 across eight tasks — math problem solving, sensitive/opinion questions, multi-hop question answering, code generation, US Medical Licensing Examination items, and visual reasoning — using an identical prompting protocol at both time points. They found large, task-dependent, and often bidirectional shifts: GPT-4 became less accurate on prime-number identification and less willing to produce directly executable code, while its performance on multi-hop knowledge questions improved; GPT-3.5 showed the reverse pattern on several tasks. The authors further isolated a plausible proximate cause, showing that GPT-4's adherence to

task-agnostic formatting instructions (e.g., “answer only with a single letter”) degraded sharply between snapshots, and argued that instruction-following drift is a common factor underlying many of the surface-level performance changes they observed (Chen et al., 2023).

Complementary evidence comes from Tu et al. (2023), who built ChatLog, a continuously updated dataset that records ChatGPT's responses to a fixed battery of 21 established NLP benchmarks on a monthly cadence, plus 1,000 fixed long-form generation prompts recorded daily. Rather than a single before/after comparison, ChatLog affords a time series, from which the authors report that most of ChatGPT's measured capabilities improve gradually over time but that some abilities move in a step-wise, non-monotonic pattern consistent with discrete model swaps rather than continuous fine-tuning drift. ChatLog additionally identifies linguistic features of model output that remain stable across versions and exploits this stability to build a more version-robust classifier for detecting machine-generated text (Tu et al., 2023) — illustrating that drift-tracking data can be repurposed for downstream applications beyond monitoring per se.

2.3 Holistic and Standardized Evaluation Frameworks

Independent of the drift-specific literature, a parallel line of work has sought to standardize LLM evaluation broadly, which bears directly on drift measurement because comparable time-series tracking requires a comparable, reproducible measurement instrument at every time point. The Holistic Evaluation of Language Models (HELM) project (Liang et al., 2022) taxonomizes the scenarios and metrics relevant to language models and evaluates dozens of models under standardized 5-shot prompting across accuracy, calibration, robustness, fairness, bias, toxicity, and efficiency, releasing all raw prompts and completions for reanalysis. The BIG-bench collaborative benchmark (Srivastava et al., 2022) similarly assembled 204 tasks contributed by hundreds of researchers to probe capabilities believed to lie beyond then-current model abilities. Neither project was designed explicitly as a drift-tracking tool, but both illustrate the infrastructure — fixed scenarios, fixed metrics, fixed prompting protocols, public release of raw completions — that longitudinal drift measurement also requires, and HELM in particular has since been re-run against successive model releases in ways that function as informal drift snapshots.

2.4 Human and LLM-Based Preference Evaluation

Much of what users experience as “behavior change” concerns qualities — tone, verbosity, helpfulness, refusal style — that are poorly captured by accuracy-style benchmarks and are instead assessed through pairwise human preference or automated LLM-as-judge protocols. Zheng et al. (2023) validate GPT-4 as a scalable proxy for human preference judgments, reporting over 80% agreement between GPT-4 and both expert and crowdsourced human raters on the MT-Bench and Chatbot Arena datasets, while cataloguing systematic judge biases (position bias, verbosity bias, limited arithmetic and reasoning ability) that must be corrected for. Any drift-tracking protocol that relies on LLM-as-judge scoring inherits these biases, and because judge models are themselves periodically updated, a drift study using an LLM judge risks confounding drift in the model under test with drift in the judge itself (Zheng et al., 2023).

2.5 Instruction Tuning, RLHF, and Behavioral Change as an Intended Effect

It is also necessary to recall that many of the same techniques implicated in unwanted drift are the techniques by which providers intentionally improve models between versions. Reinforcement learning from human feedback (RLHF), as formalized in the InstructGPT line of work (Ouyang et al., 2022), is explicitly designed to change model behavior — improving

instruction-following and reducing harmful outputs — relative to the preceding checkpoint. Viewed this way, drift is not simply an engineering defect but the visible trace of an active, continuing alignment and safety-tuning process; the empirical challenge is to distinguish desirable behavioral change (e.g., reduced toxicity) from undesirable capability regression (e.g., degraded arithmetic accuracy) within the same update, a distinction the existing literature draws only partially (Chen et al., 2023; Ouyang et al., 2022).

3. A Taxonomy of Drift Sources and Manifestations

Because “LLM drift” is used loosely in the literature to cover heterogeneous phenomena, this section proposes a working taxonomy organized by mechanism, distinguishing sources that are internal to the model's parameters from those that are external to it but still visible to the end user.

3.1 Weight-Level Drift: Retraining and Fine-Tuning

The most direct source of drift is a change to the model's parameters themselves — continued pre-training on fresh data, supervised fine-tuning on new instruction data, or an additional round of RLHF. Fine-tuning updates of this kind are known to induce catastrophic forgetting, in which gains on a new task or objective come at the cost of previously acquired knowledge or skills. Luo et al. (2023) empirically demonstrate that catastrophic forgetting is prevalent in continual instruction tuning of LLMs ranging from one to seven billion parameters, that the severity of forgetting increases with model scale, and that architecture (decoder-only versus encoder-decoder) and the diversity of prior instruction tuning both modulate how much knowledge is retained. Applied to versioning, this suggests that later, more heavily instruction-tuned or safety-tuned model versions are not guaranteed to be strict supersets of their predecessors' capabilities: a later version may be more aligned and more helpful on average while simultaneously having forgotten specific factual or procedural competencies the earlier version possessed (Luo et al., 2023).

3.2 System-Level Drift: Prompts, Safety Filters, and Serving Infrastructure

A second class of drift occurs without any change to model weights at all. Providers frequently alter the hidden system prompt, add or adjust content-moderation layers, change default sampling parameters, or swap the serving stack (e.g., quantization scheme, batching strategy, hardware backend) in ways that alter observed outputs for a nominally unchanged model. Industry practitioner analyses have specifically flagged system-prompt and moderation-layer changes as a major contributor to what end users perceive as capability loss, noting that the June 2023 update evaluated by Chen et al. (2023) coincided with documented changes to default system behavior (Chen et al., 2023). From a user's vantage point, weight-level and system-level drift are indistinguishable without provider disclosure, which is one reason the literature increasingly favors the model-agnostic term “behavior drift” over language that presumes a particular cause.

3.3 Evaluation-Level Drift: Contamination and Benchmark Decay

A third, more subtle source of apparent drift is not a change in the model at all but a change in the validity of the measuring instrument. As benchmark datasets circulate on the web, they risk being scraped into the pre-training corpora of subsequent model versions, a phenomenon termed benchmark data contamination. Xu, Guan, Greene, and Kechadi (2024) survey this literature, defining contamination as the inadvertent incorporation of evaluation data into training data, and cataloguing detection methods (n-gram overlap, membership inference, perturbation-based tests) and mitigation strategies (dynamic or continually refreshed benchmarks, canary strings, decontamination pipelines). For drift tracking specifically, contamination creates an asymmetric risk: a later model version may appear to

“improve” on a static benchmark not because its underlying capability increased but because that benchmark's answers entered its training data after the earlier version was released — a confound that a naive before/after benchmark comparison cannot distinguish from genuine capability gain (Xu et al., 2024).

3.4 Task-Dependent Heterogeneity of Drift

A consistent finding across the empirical studies reviewed here is that drift is not a single scalar quantity but a vector that can point in different directions on different tasks within the same version transition. Chen et al. (2023) report simultaneous improvement and degradation across their eight evaluated tasks for both GPT-3.5 and GPT-4; Tu et al. (2023) similarly find that most ChatGPT capabilities improve over their monitoring window while a minority regress, producing what they describe as a step-wise, non-uniform evolution pattern. This heterogeneity has a direct practical implication: an aggregate, single-number “model quality” score is an inadequate unit of analysis for drift tracking, because it can mask offsetting task-level gains and losses; disaggregated, task- and capability-specific tracking, in the spirit of HELM's multi-metric approach (Liang et al., 2022), is necessary to characterize drift meaningfully.

4. Current Approaches and Developments in Drift Monitoring

4.1 Longitudinal Benchmark Replays

The dominant current methodology is the longitudinal replay: administer a fixed set of prompts to successive model versions and compare outputs quantitatively. LLMDrift (the dataset and evaluation harness released alongside Chen et al., 2023) exemplifies the minimal version of this approach — a two-point-in-time comparison across a curated task battery, with prompts and completions released publicly for independent reanalysis. ChatLog extends the same logic to a continuous, higher-frequency cadence: monthly snapshots across 21 established NLP benchmarks and daily snapshots of long-form generation on a fixed prompt set, which allows the shape of drift (gradual versus abrupt) to be characterized rather than merely its endpoints (Tu et al., 2023). Extending fixed-prompt replay to a continuous cadence is valuable precisely because it can reveal whether a change is a discrete step (consistent with a new model deployment) or a gradual trend (consistent with continuous fine-tuning or shifting moderation thresholds).

4.2 Holistic, Multi-Metric Evaluation Reused for Tracking

HELM's design — an explicit taxonomy of scenarios and metrics, standardized prompting, and public release of raw model completions — was motivated by cross-model transparency rather than drift per se, but the same infrastructure is directly reusable for tracking a single model family across its own successive releases, since HELM evaluates “all 30 models... on all 42 scenarios” under identical standardized conditions (Liang et al., 2022). Because HELM measures seven dimensions (accuracy, calibration, robustness, fairness, bias, toxicity, efficiency) rather than accuracy alone, it is structurally well suited to revealing the kind of multidimensional, offsetting drift described in Section 3.4, where a version update might reduce toxicity while also reducing calibration or increasing latency.

4.3 Preference-Based and LLM-as-Judge Tracking

Because much of what users mean by “the model got worse” concerns stylistic and preference-sensitive qualities rather than benchmark accuracy, several monitoring efforts supplement accuracy-style replay with pairwise preference comparison. Chatbot Arena-style crowdsourced battles and MT-Bench-style LLM-as-judge scoring (Zheng et al., 2023) can, in principle, be run against successive versions of the same underlying model to produce a win-rate trend line analogous to an Elo rating over time, capturing helpfulness

and conversational quality in a way accuracy benchmarks do not. The same paper's documentation of judge biases (position, verbosity, self-enhancement) is itself directly relevant to drift tracking: if the judge model is also being updated over time, apparent preference drift in the model under test may partly reflect drift in the judge's evaluation criteria (Zheng et al., 2023).

4.4 Documentation and Disclosure Mechanisms

A complementary, non-empirical current approach is structured documentation. Mitchell et al. (2019) introduced model cards as short, standardized documents accompanying a released model, recording basic information (developer, date, version, training algorithms), intended use cases, evaluation factors, and disaggregated quantitative results. Although model cards were designed principally for fairness and appropriate-use disclosure rather than drift tracking, a versioned series of model cards — one per release, each reporting the same disaggregated metrics — would itself constitute a drift record, and several providers have begun publishing incremental “system card” updates alongside major releases in a manner that partially fulfills this function, though without the standardized, comparable metric sets that would allow rigorous cross-version statistical comparison (Mitchell et al., 2019).

4.5 User- and Practitioner-Driven Monitoring

Outside the peer-reviewed literature, a substantial amount of drift detection currently happens informally: individual developers and small teams notice that a previously reliable prompt has stopped producing the expected output format and report this anecdotally, as documented in contemporaneous journalistic and practitioner coverage of the Chen et al. (2023) findings. Commercial LLM-observability tooling has emerged to formalize this practice, offering regression test suites that re-run a team's own production prompts against a new model version before it is adopted and flagging output drift on production traces in real time. While these tools operationalize the same fixed-prompt-replay logic as the academic studies reviewed above, they typically evaluate against a single organization's own prompt distribution rather than a shared, publicly reproducible benchmark, limiting their contribution to generalizable scientific knowledge about drift even as they provide practical value to the teams that use them.

5. Research Challenges and Limitations

5.1 Confounding of Weight, System, and Measurement Drift

As discussed in Section 3, an external observer generally cannot distinguish a change in model weights from a change in the surrounding system prompt, moderation layer, or serving stack, nor from a change in the validity of the benchmark itself due to contamination. Existing empirical studies acknowledge this directly: Chen et al. (2023) note that because model providers do not disclose what changed between snapshots, their study can document that behavior changed and offer instruction-following drift as one plausible common factor, but it cannot fully attribute observed changes to a specific engineering cause. This is a fundamental limitation of black-box API-based drift studies as currently practiced.

5.2 Benchmark Contamination and Validity Decay

The contamination literature surveyed by Xu et al. (2024) implies that any fixed, static benchmark used repeatedly for longitudinal tracking has a finite shelf life: the more publicly it circulates (including, ironically, through its use in prior drift studies whose data and prompts are released openly, as both LLMDrift and ChatLog do), the more likely it is to leak into the training data of subsequent model versions, inflating later scores independent of

genuine capability change. This creates a tension between the transparency norms of open science, which favor public release of evaluation prompts and completions, and the measurement validity requirements of drift tracking, which favor withholding or continually rotating evaluation material.

5.3 Non-Determinism and Sampling Variance

LLM outputs are stochastic even for a fixed model version, prompt, and temperature setting, both because of sampling randomness and because of documented sources of hardware- and batching-level non-determinism in production serving stacks. A drift study that compares a single completion per prompt at two time points risks mistaking within-version sampling variance for genuine cross-version drift, particularly on tasks with small absolute accuracy differences; robust drift claims require repeated sampling and explicit statistical testing analogous to the sequential change-detection methods developed in the classical concept-drift literature (Gama et al., 2014), a standard not uniformly met by early LLM-specific drift studies.

5.4 Judge and Evaluator Drift

Where drift measurement relies on an LLM-as-judge protocol, the evaluator itself is subject to the same versioning opacity as the model under test. Zheng et al. (2023) document that judge models carry systematic biases (position, verbosity, self-enhancement, limited multi-step reasoning) that are unlikely to be perfectly stable across the judge's own version updates. A longitudinal study that uses, say, successive GPT-4 snapshots to grade successive Claude snapshots is measuring a moving target with another moving target, and the field currently lacks agreed methodology for separating these two sources of variance.

5.5 Absence of Standardized Statistical Protocols

The classical concept-drift literature treats drift detection as a formal hypothesis-testing problem, with defined false-positive control and a vocabulary distinguishing sudden, gradual, incremental, and recurring drift patterns (Gama et al., 2014). The LLM-drift literature to date is largely descriptive: studies report percentage-point differences in accuracy or agreement between two or a handful of time points without formal significance testing, confidence intervals, or an explicit drift-detection algorithm. This makes it difficult to compare the magnitude of drift reported across different studies, tasks, or model families, or to set a principled threshold above which a change should be considered actionable drift rather than noise.

5.6 Restricted Access and External Reproducibility

Because the most widely used commercial LLMs are accessible only through APIs controlled by their providers, independent researchers cannot inspect model weights, training data, or infrastructure changes directly; they can only probe behavior through the same input-output interface available to any user. This constrains drift research to black-box methods and means that provider-side disclosures (or their absence) substantially determine what can be learned. It also means that a provider could, in principle, alter a model in response to a published drift study — whether to address the finding or to obscure it — in a way that undermines the reproducibility of the original study's snapshot, a risk inherent to studying externally hosted, continuously updated commercial systems rather than fixed research artifacts.

6. Research Gaps and Future Directions

6.1 Contamination-Resistant, Continuously Refreshed Benchmarks

A clear gap is the near-absence of benchmarks purpose-built for longitudinal drift tracking that are also resistant to the contamination dynamics described in Section 5.2. Dynamic benchmark generation — programmatically producing fresh problem instances from a fixed template distribution, or rotating held-out item pools on a schedule tied to model release dates — would allow each snapshot in a time series to be evaluated on material that could not have contaminated the training data of the version being tested. Building such infrastructure specifically for drift tracking, rather than for one-off cross-model leaderboards, is an underexplored engineering and research opportunity that would draw on both the contamination-mitigation literature (Xu et al., 2024) and the dynamic-benchmarking impulse already visible in general LLM evaluation research.

6.2 Formal Drift-Detection Statistics for Generative Models

The classical concept-drift literature's toolkit of sequential change-point detectors, confidence-bounded error tracking, and explicit sudden/gradual/incremental/recurring taxonomy (Gama et al., 2014) has not yet been systematically adapted to the generative, multi-metric, partially subjective evaluation setting that characterizes LLMs. Future work could define drift-detection statistics appropriate to LLM output distributions (e.g., over token-level probability, semantic-similarity, or preference-win-rate time series) with explicit false-positive control, enabling a principled answer to the question “should this observed change be flagged as drift?” rather than the current reliance on eyeballed percentage-point comparisons.

6.3 Causal Attribution of Drift to Its Source

Because current studies cannot distinguish weight-level, system-level, and measurement-level drift (Section 5.1), a valuable research direction is developing indirect diagnostic tests that narrow down the likely cause without provider cooperation — for instance, probing whether observed changes correlate more strongly with alterations to formatting-sensitive instructions (suggestive of system-prompt or safety-layer changes) than with alterations to substantive task content (suggestive of weight-level change), building on the instruction-following diagnostic already piloted by Chen et al. (2023). Parallel work encouraging or mandating provider-side disclosure, modeled on the model-card framework (Mitchell et al., 2019) but extended to versioned, comparably structured update logs, would reduce the field's dependence on such indirect inference.

6.4 Disaggregated, Multidimensional Drift Reporting

Given the demonstrated heterogeneity of drift across tasks (Section 3.4), future benchmarking efforts should adopt HELM-style multi-metric, disaggregated reporting (Liang et al., 2022) as the default unit of drift analysis rather than a single aggregate score, and should explicitly report both the direction and magnitude of change per task and per metric across versions, enabling downstream users to identify whether a specific capability they depend on is likely to have shifted, rather than relying on an aggregate signal that can mask offsetting gains and losses.

6.5 Joint Study of Catastrophic Forgetting and Deployment-Time Drift

The catastrophic-forgetting literature (Luo et al., 2023) and the deployment-time behavior-drift literature (Chen et al., 2023; Tu et al., 2023) have developed largely in parallel, the former in controlled fine-tuning experiments on open models, the latter through black-box probing of commercial APIs. Bridging these lines of work — for instance, by studying whether the forgetting patterns identified in controlled academic fine-tuning experiments

predict the specific capability regressions observed in commercial version transitions — would connect a mechanistic account of why drift occurs to the black-box observational record of when and how much it occurs in deployed systems.

6.6 Governance, Reproducibility Norms, and Versioned Disclosure

Finally, there is a policy and norms gap: neither academic publishing conventions nor industry practice currently require that a benchmark result be accompanied by a durable, retrievable snapshot of the exact model version tested, an explicit re-test protocol for later verification, or a commitment to timestamped, versioned system documentation. Extending the model-card proposal (Mitchell et al., 2019) into a living, versioned document — updated and re-published at each model revision, with consistent disaggregated metrics across revisions — would give the research community, regulators, and practitioners a shared, comparable record against which independent drift studies could be validated, reducing reliance on the kind of after-the-fact, black-box reverse engineering that characterizes the current literature.

7. Conclusion

The empirical record assembled over the past several years — beginning with Chen et al.'s (2023) demonstration that GPT-3.5 and GPT-4 changed substantially and non-uniformly across a single three-month interval, extended by Tu et al.'s (2023) continuous ChatLog monitoring, and framed by longer-standing evaluation infrastructure such as HELM (Liang et al., 2022) and LLM-as-judge methodology (Zheng et al., 2023) — establishes performance drift as a real, measurable, and practically consequential feature of deployed large language models, not merely a subjective impression. At the same time, this review has shown that the field's methodological toolkit remains considerably less mature than the object of study demands: drift studies to date have generally been descriptive rather than statistically rigorous, cannot cleanly separate weight-level change from system-level or measurement-level confounds such as benchmark contamination (Xu et al., 2024), and often rely on evaluation instruments — fixed benchmarks, LLM judges — that are themselves not immune to the very instability being studied. The older concept-drift literature in machine learning (Gama et al., 2014) and the documentation norms pioneered by model cards (Mitchell et al., 2019) offer conceptual and institutional resources that the LLM-drift literature has only begun to draw upon. Closing this gap — through contamination-resistant continuous benchmarks, formal drift-detection statistics, causal attribution methods, and versioned disclosure norms — would allow the field to monitor the evolving behavior of deployed language models with a rigor commensurate with how much downstream infrastructure, research, and public understanding now depends on that behavior remaining, or being known not to remain, the same from one version to the next.

References

Chen, L., Zaharia, M., & Zou, J. (2023). How is ChatGPT's behavior changing over time? arXiv. arXiv:2307.09009. https://doi.org/10.48550/arXiv.2307.09009

Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A survey on concept drift adaptation. ACM Computing Surveys, 46(4), Article 44, 1–37. https://doi.org/10.1145/2523813

Liang, P., Bommasani, R., Lee, T., Tsipras, D., Soylu, D., Yasunaga, M., Zhang, Y., Narayanan, D., Wu, Y., Kumar, A., Newman, B., Yuan, B., Yan, B., Zhang, C., Cosgrove, C., Manning, C. D., Ré, C., Acosta-Navas, D., Hudson, D. A., Zelikman, E., … Koreeda, Y. (2022/2023). Holistic evaluation of language models. arXiv:2211.09110. Also published in Transactions on Machine Learning Research, 2023. https://doi.org/10.48550/arXiv.2211.09110

Luo, Y., Yang, Z., Meng, F., Li, Y., Zhou, J., & Zhang, Y. (2023). An empirical study of catastrophic forgetting in large language models during continual fine-tuning. arXiv:2308.08747. https://doi.org/10.48550/arXiv.2308.08747

Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., & Gebru, T. (2019). Model cards for model reporting. In Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT* '19) (pp. 220–229). Association for Computing Machinery. https://doi.org/10.1145/3287560.3287596 (arXiv:1810.03993)

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., & Lowe, R. (2022). Training language models to follow instructions with human feedback. Advances in Neural Information Processing Systems, 35, 27730–27744. arXiv:2203.02155

Polyportis, A. (2024). A longitudinal study on artificial intelligence adoption: Understanding the drivers of ChatGPT usage behavior change in higher education. Frontiers in Artificial Intelligence, 6, 1324398. https://doi.org/10.3389/frai.2023.1324398

Srivastava, A., Rastogi, A., Rao, A., Shoeb, A. A. M., Abid, A., Fisch, A., Brown, A. R., Santoro, A., Gupta, A., Garriga-Alonso, A., … Wu, Z. (2022). Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. arXiv:2206.04615. Also published in Transactions on Machine Learning Research, 2023.

Tu, S., Li, C., Yu, J., Wang, X., Hou, L., & Li, J. (2023). ChatLog: Recording and analyzing ChatGPT across time. arXiv:2304.14106. https://doi.org/10.48550/arXiv.2304.14106

Xu, C., Guan, S., Greene, D., & Kechadi, M.-T. (2024). Benchmark data contamination of large language models: A survey. arXiv:2406.04244. https://doi.org/10.48550/arXiv.2406.04244

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. Advances in Neural Information Processing Systems 36 (NeurIPS 2023 Datasets and Benchmarks Track). arXiv:2306.05685